

THE UNIVERSITY OF BRITISH COLUMBIA
Electrical and Computer Engineering Department

CPEN 442

Introduction to Cybersecurity

Winter Term 1, 2026

ASSIGNMENT 1: ANCIENT CRYPTO

Due date: **Friday, Sep 25th, 2026, 11:59 pm**
Hard deadline: **Sunday, Sep 27th, 2026, 11:59 pm**

Use Piazza for questions. Instructor and TA support ends at 2 PM on the due date. A 48-hour automatic extension is provided for upload issues. There is no penalty until the hard deadline; no submissions accepted after that date. For long-term issues, contact the instructor *before the due date*.

Total marks: 100 pts

Doing the Assignment: This assignment must be completed individually or in groups of two. You may discuss it with others, but you must not *copy* or share solutions (this includes not having a public repository with your solutions for the duration of the course).

Generative AI Disclosure. You may use generative AI for any part of this assignment and will not be penalized for doing so. However, use it as a tool rather than a substitute for understanding. If you use generative AI, include a final section titled “Generative AI Disclosure” describing how and where it was used. This section must be *between half a page and one page long*, and cannot be written by AI.

What to submit

- A pdf file called **a1.pdf**, with the answers to questions Q1, Q2, Q3, and AI disclosure (if applicable). You can create this with Microsoft Word, Google Docs, a text editor, or latex. Just make sure it is clear where each answer is, and that the pdf is readable.

Preamble

The goal of this assignment is to understand why ancient ciphers are *weak* and how they can be *broken*. A key takeaway is that the *appearance of encryption does not guarantee security*. We will work with five ciphers discussed in class: the Caesar cipher, the Vigenère cipher, the affine cipher, the Playfair cipher, and the One-Time Pad (OTP).

Before you start, decide if you will do this assignment individually or paired with someone else, and join an “Assignment Group” on Canvas accordingly. Your group number decides which ciphertexts you have to break, so do this *before* you start working: if you join a different group later, your ciphertexts change. Then, download the assignment files. You will find the following.

Assignment Files

- The file `ciphertexts.zip`, which contains 80 files with ciphertexts, numbered 1 to 80.
- The files `plaintexts_0.txt`, `ciphers_and_keys_0.txt`, and `ciphertexts_0.txt`, which contain a worked example for debugging purposes.

Go inside `ciphertexts.zip` and take a look at `ciphertexts_0.txt`, where i is your group number. You will see six strings, one per line of the document. Each of these strings is a ciphertext that has been encrypted with an ancient cipher. Your primary goal in this assignment is to guess the plaintexts.

The following explains how we have generated the plaintexts and ciphertexts.

1. First, we asked ChatGPT to generate 151 plaintexts briefly describing each of the first 151 Pokémon, using only letters, commas, and periods. Each plaintext always begins with a Pokémon name (this is something ChatGPT did by default, and we left it like this. Note that this is a known plaintext attack!).
2. For each of these plaintexts, we processed them by converting all letters to upper-case ASCII characters (from A to Z, i.e., 26 possible characters), and deleted all other characters (spaces, commas, and periods).
3. For each student group, we sampled *six* of these plaintexts at random, and encrypted them with the following ciphers:
 - **Caesar cipher.** We chose the key as a random integer from 1 to 25 that denotes the letter shift from plaintext to ciphertext.
 - **Affine cipher.** The key is the tuple (a, b) , where $a, b \in [0, 1, \dots, 25]$. We sampled keys randomly from the space of all possible keys (note: not all values of a are “possible” keys, since a^{-1} must exist, as seen in the classroom).

- **Vigenère cipher.** This works exactly as we have seen in the classroom. We asked ChatGPT to generate English words at random to use as keys.
 - **Playfair cipher.** Starting with the plaintexts with letters [A-Z], we replaced all J characters with I, then we added an X between *any* two consecutive identical characters. We made sure the length of the plaintext was even and at most 300 characters. Then, we encrypted the plaintext. The key for the Playfair cipher is a random permutation of [A-Z], excluding J. Thus, the key has a length of 25 characters.
 - **One-Time Pad (OTP).** To run the OTP, we first converted each character into 5 bits using the RFC 4648 Base32 standard. Then, we XORed it with a (pseudo-)random key.¹ Then we converted the ciphertext bits back to characters using the RFC 4648 Base32 standard.
4. After encrypting your plaintexts, we wrote them to the `ciphertexts_i.txt` file that you have, in a random order (e.g., the Caesar cipher could be in any of the six lines).
 5. *Note:* we used the OTP twice, with the same key. This explains why you see six ciphertexts instead of five.

Examples

To help you understand how your ciphertexts were generated, you have one example with plaintexts available. Group number 0 is not assigned to anybody, so these files do not give away anybody's answers. The file `plaintexts_0.txt` contains six plaintexts, one per line. The file `ciphers_and_keys_0.txt` contains the names of the ciphers followed by their key (six lines, one per cipher and key). The file `ciphertexts_0.txt` contains six ciphertexts, one per line. The ciphertext in line j ($j \in [6]$) has been generated by taking the j th plaintext in `plaintexts_0.txt` and encrypting it with the cipher and key in the j th line of `ciphers_and_keys_0.txt`.

¹This means it is not exactly an OTP, but for the purposes of this assignment it does not matter.

1 The Index of Coincidence (IC)

In the lectures, we saw the Index of Coincidence (IC), which is a metric that measures how likely is that, if you picked two letters at random from a text, both are the same letter.

If n_A denotes the number of times the character A appears in the text, n_B denotes the number of times the character B appears in the text, and so on, then the IC is computed as

$$IC = c \cdot \left(\frac{n_A}{N} \cdot \frac{n_A - 1}{N - 1} + \frac{n_B}{N} \cdot \frac{n_B - 1}{N - 1} + \dots \right). \quad (1)$$

Here, c is the number of letters in the alphabet (in our case, for English, $c = 26$), and N is the length of the text.

For example, if the text was ABAABC, the IC would be

$$IC = 26 \cdot \left(\frac{3}{6} \cdot \frac{2}{5} + \frac{2}{6} \cdot \frac{1}{5} + 0 \right). \quad (2)$$

[20 pts] Question 1: For *each* of the ciphers excluding OTP (Caesar, Vigenère, affine, and Playfair), explain whether the IC of the ciphertext generated with that cipher is expected to be higher, lower, or equal to the plaintext's IC. Write your answer in the file `a1.pdf`. You do not need code to do this, but you are free to program the IC function yourself to verify that your intuition is correct.

Make sure you *understand* why, as you might be asked about this in an exam. This could be used, for example, to detect a cipher, and it can also help guess the key length of a Vigenère cipher.

2 Detecting Ciphers

[30 pts] Question 2: Consider the ciphertexts in file `ciphertexts_i.txt`, where i is your assignment group number. One of them is a Caesar cipher, one is an affine cipher, one is a Vigenère cipher, one is a Playfair cipher, and two are an OTP. *Without breaking (decrypting) them*: figure out which ciphertext corresponds to which cipher. For each cipher, you must explain what you did to figure this out (note that this is possible to do without cracking the key; you can use the IC, compute histograms, etc.). Write your answer in the file `a1.pdf`, including anything that has helped you detect which cipher is what (e.g., screenshots of prints, figures you generated, etc). If you detect it with manual inspection, that is OK, but explain how you did it. Detecting “by elimination” is also valid.

3 Breaking Ciphers

Now that you know which cipher has generated which ciphertext, it is time to break them. For this assignment, you only need to guess the name of the Pokémon that is described in each plaintext. As always, you can program some code yourself or ask an LLM. Additionally, for this question only: you can use online solvers. Whatever you do: make sure you understand how the attack works.

[50 pts] Question 3: In [a1.pdf](#), for each ciphertext, provide the name of the Pokémon that is described in the plaintext and *explain* how you guessed this. The points for this question depend on how many correct names (with correct explanations) you report. If you cannot break a cipher, randomly-guessing the Pokémon name will not count as a success.

Correct Pokémon Names	Points
1	10
2	20
3	30
4	40
5	47
6	50

Table 1: Grading scheme for this question.

Note: reporting five names is almost the same points as reporting all six. The reason for this is that one of the ciphers is particularly time-consuming to break, and it might not be worth doing it. But all of this is doable.